Generating Board Games with MCTS and LLM Agents

Gabriel Bauer de Oliveira, Anderson Rocha Tavares

Instituto de Informática – Universidade Federal do Rio Grande do Sul (UFRGS)
Caixa Postal 15.064 – 91.501-970 – Porto Alegre – RS – Brazil
{gbboliveira, artavares}@inf.ufrgs.br

Abstract

Large Language Models (LLMs) have shown significant potential in Procedural Content Generation (PCG). Yet, their application to the generation and balancing of interactive game mechanics remains largely unexplored. While PCG is a mature field, most research focuses on static assets rather than executable mechanics. Existing systems for rule synthesis frequently utilize domain-specific languages, e.g., the Ludii Game Description Language (Ludii GDL), which impose structural constraints. By contrast, generating games in Python leverages the extensive pre-training data of LLMs. To address this gap, we introduce an agentic pipeline for the procedural generation and dynamic balancing of board game mechanics. From a natural language prompt, a programmer LLM synthesizes executable rulesets. An MCTS agent then runs playouts to extract viability metrics, such as completion, balance, agency, and coverage. These metrics are compared against classic strategy baselines, and a Mutation Agent uses the largest deviation to guide targeted rule or code mutations. Our evaluation demonstrates that this metric-informed, closed-loop approach effectively optimizes the procedural search space, allowing the system to iteratively refine and balance playable game mechanics.

1 Introduction

Large Language Models (LLMs) have recently demonstrated the ability to synthesize executable code directly from natural-language specifications (Jiang et al. 2024), prompting growing interest in applying them to domains long used as benchmarks for artificial intelligence. Board games are a prominent example: they have driven advances in search, planning, and learning algorithms for decades (Shannon 1950; Campbell, Jr., and hsiung Hsu 2002; Silver et al. 2016, 2018), and the shift from single-shot prompting toward agentic systems – in which specialized agents collaborate through planning, tool use, and iterative refinement (Wang et al. 2025; Xi et al. 2023; Jiang et al. 2026) – has now extended this interest specifically to game AI, where LLMs are increasingly applied to procedural content generation (PCG), mixed-initiative design, and autonomous game development (Yang, Kleinman, and Harteveld 2024, 2025).

Most of this work, however, targets content (e.g. levels, narratives, assets) inside a fixed ruleset (Yang, Kleinman, and Harteveld 2025). The mechanics, i.e., rules of state transitions and victory conditions, remain largely untouched. This is a harder problem than typical content generation, since correctly-implemented mechanics can still represent a game that is unbalanced, trivial, or strategically flat – properties that only become apparent once the game is actually played (Browne 2008).

Most systems that address mechanics generation encode games in a domain-specific description language (e.g. (Piette et al. 2020)), which constrains expressiveness and requires the model to operate within a specialized grammar on top of the task itself. Using general-purpose code instead sidesteps that constraint – modern LLMs have seen far more ordinary source code than any game description language during pre-training (Jiang et al. 2024) – albeit stopping at one-shot generation, with no mechanism for iterative refinement (Becker et al. 2025). How an LLM can iteratively refine a game's mechanics using only gameplay feedback remains an open direction.

This paper addresses that problem through an agentic pipeline to generate board games as executable Python code from natural-language descriptions and iteratively refines them through repeated evaluation and gameplay-driven feedback. A sequence of specialized agents – formalizing rules, planning an implementation, and generating and validating code against the Boardwalk 2.0 API – produces a complete candidate, which is then evaluated through Monte Carlo Tree Search (MCTS) self-play against a suite of classic strategy games. A *Mutation Agent* identifies the metric that deviates most from established, well-balanced play and proposes a targeted rule or code change; only candidates that quantitatively improve the fitness score or overcome a fundamental structural failure are retained, closing the loop from prompt to gameplay-driven feedback.

Code execution alone is not treated as sufficient evidence of success in this framework; each candidate is judged by how it behaves during play. This combination of automated coding, validation, self-play evaluation, and refinement allows the system to improve game mechanics without human intervention after the initial prompt.

This work is guided by the following research questions:

- **RQ1.** How does the level of detail in the initial prompt influence both the baseline quality and the efficacy of metric-guided iterative refinement in LLM-generated

Copyright © 2026, Association for the Advancement of Artificial Intelligence (www.aaai.org). All rights reserved.

board games?

- **RQ2.** How consistently does the proposed pipeline perform across different language models and providers?

The main contributions of this work are:

- an end-to-end agentic pipeline for generating, validating, evaluating, and iteratively refining board game mechanics represented as general-purpose Python code;
- a closed-loop refinement strategy that combines MCTS-based self-play with quantitative gameplay metrics to guide LLM-generated rule mutations;
- an extension of the Boardwalk framework that introduces immutable game states and supports tree search, enabling automated gameplay and iterative refinement.

2 Related Work

Recent advances in Large Language Models (LLMs) have transformed game AI research, enabling increasingly sophisticated applications across procedural content generation, game design, gameplay assistance, and automated development. Surveys by Yang *et al.* (Yang, Kleinman, and Harteveld 2024, 2025) categorize this rapidly growing literature into procedural content generation (PCG), mixed-initiative game design and development (MIGDD), mixed-initiative gameplay (MIG), autonomous gameplay, and game user research. This work lies primarily at the intersection of procedural content generation and automated game design, while drawing on ideas from gameplay evaluation and agentic LLM systems.

2.1 LLMs for Game Generation

Most LLM-based game generation research focuses on producing content within an existing game rather than creating entirely new game mechanics. Representative examples include MarioGPT (Sudhakaran et al. 2023), controllable Sokoban level generation (Todd et al. 2023), and narrative or card content for specific existing games (Zammit and Liapis 2024; Lyman et al. 2024); these approaches show LLMs are effective at creating levels, narratives, and other assets, but operate within rules that remain fixed and human-designed.

A complementary direction uses LLMs as collaborative design assistants, supporting designers through brainstorming and iterative refinement (Lanzi and Loiacono 2023; Anjum, Li, and Togelius 2024; Torii, Murakami, and Ochiai 2023), including multi-agent frameworks that decompose development into specialized tasks (Chen et al. 2023); human designers, however, remain responsible for evaluating gameplay quality. More broadly, agentic LLM workflows — decomposing problems into specialized agents for planning, implementation, and reflection — have proven effective for software engineering (Hong et al. 2024; Qian et al. 2024; Wu et al. 2024; Li et al. 2023; Wang et al. 2025) and, more recently, procedural content generation (Jiang et al. 2026), though these systems generally assist human designers rather than autonomously improving mechanics based on gameplay behavior.

2.2 Executable Game Mechanics Generation

Generating the mechanics themselves remains considerably less explored than generating game assets. Existing approaches can be broadly divided into two representational paradigms.

The dominant direction represents games using domain-specific game description languages. Ludii (Piette et al. 2020) has become the most widely adopted representation for this purpose due to its expressive grammar and support for automated gameplay. Building upon this representation, GAVEL (Todd et al. 2024) combines MAP-Elites with LLM-driven mutation operators to generate diverse board games, while other work synthesizes games through grammar-based descriptions (Tanaka and Simo-Serra 2024), the Video Game Description Language (VGDL) (Ebner et al. 2013; Schaul 2013; Perez-Liebana et al. 2016; Hu, Zhao, and Liu 2024), or explicit VGDL rule generation (Khalifa et al. 2017). These approaches successfully generate executable games, but require language models to reason within specialized formalisms that differ substantially from the general-purpose source code predominantly encountered during pretraining.

An alternative direction generates game mechanics directly as executable source code. Cook's line of work pioneered this approach: Mechanic Miner (Cook et al. 2013) uses reflection to search a platformer's own codebase for candidate fields and pair them with a small set of predefined modifier operations to form toggleable player abilities, evaluating each through simulated play. Pixie (Cook 2025) extends this idea to modern Unity projects, using evolutionary search to generate and compile freeform C# code snippets that are installed directly into a game's existing codebase against a designer-specified objective. Both systems, however, operate on small, isolated mechanic fragments inserted into a pre-existing game.

Moving from isolated snippets to complete rulesets, Boardwalk (Becker et al. 2025) demonstrated that modern LLMs can synthesize complete board games as Python programs using a lightweight programming interface, leveraging their extensive exposure to general-purpose code during training (Jiang et al. 2024). Representing games as ordinary source code avoids the representational limitations of domain-specific languages while improving accessibility for developers. A recent benchmark evaluates LLM-driven program search for games across multiple representations and languages (Eberhardinger et al. 2025), but, like Boardwalk, does not close the loop with gameplay-driven refinement. Nevertheless, existing Python-based approaches remain fundamentally one-shot: generated games are evaluated manually, and no automated mechanism exists for refining mechanics through repeated gameplay feedback — the gap our pipeline addresses.

2.3 Automated Gameplay Evaluation

Long before the emergence of LLMs, automated gameplay evaluation had become an established technique for assessing the quality of generated games, supported by general-purpose tabletop-game AI infrastructure such as the TAG

framework (Gaina et al. 2020). Evolutionary game design uses simulated play to estimate properties such as balance, completion rate, strategic diversity, and player advantage (Browne 2008). An early example is Hom and Marks (Hom and Marks 2007), who used genetic algorithms to optimize draw rate and first-player advantage by mutating rules in a general game-definition language rather than executable code. Similar ideas have been applied to AI-assisted board game design (Goodman et al. 2023), automated balancing (Beyer et al. 2016), industrial playtesting (Zhao et al. 2020), contemporary board-game playtesting through agent performance profiling (de Mesentier Silva et al. 2017), and gameplay analysis using heterogeneous agent teams (Guerrero-Romero, Lucas, and Perez-Liebana 2018). Collectively, these studies demonstrate that simulated gameplay provides meaningful quantitative signals that correlate with desirable gameplay characteristics.

The present work builds on this tradition of automated gameplay evaluation, but integrates it directly into an agentic generation pipeline rather than treating it as an external evaluation tool. Taken together, the two representational paradigms reviewed above leave a gap that neither closes alone: domain-specific systems such as GAVEL benefit from automated, evolutionary refinement, but require reasoning within a specialized formalism; general-purpose-code approaches such as Boardwalk offer the flexibility of ordinary programming languages, but stop at one-shot generation, with no mechanism for iterative improvement. Our pipeline is designed to combine both advantages: it generates board games as executable Python programs, and closes the refinement loop by using gameplay metrics extracted from Monte Carlo Tree Search (MCTS) self-play to guide successive rule mutations — bridging two research directions that have so far evolved largely independently.

3 Methods

This section describes the proposed agentic pipeline: its architecture, the Boardwalk 2.0 game representation, each generation and refinement stage, the evaluation metrics and baseline suite, the mutation and acceptance procedure, and the prompt suite used in our experiments.

3.1 System Overview

The proposed pipeline treats board game generation as a closed-loop process run by specialized LLM agents, not a single synthesis step. Figure 1 shows the architecture. From a natural-language prompt, the pipeline produces a rule specification, an implementation plan, executable Boardwalk 2.0 code, and checks both its static correctness and runtime behavior. Validated games are evaluated through MCTS self-play; the resulting metrics are compared against a baseline suite and used to guide targeted mutations. The mutated implementation re-enters validation and evaluation, closing the loop until a stopping criterion is reached.

The pipeline runs in two phases. The generation phase turns the prompt into an initial executable implementation through rule formalization, planning, code generation, and validation. The refinement phase then repeatedly evaluates the current best candidate, proposes a targeted mutation, and re-validates and re-evaluates the result.

Every run begins from a natural-language prompt alone, with no seed game or reference implementation supplied; the first candidate the *Coding Agent* produces is therefore a complete, from-scratch implementation, and, since no prior candidate exists for comparison, Algorithm 1 accepts it unconditionally ($f_{\text{best}} = -\infty$) before mutation begins.

The pipeline keeps a single elite solution throughout the run: a new candidate replaces it only by meeting the acceptance criteria in Section 3.6, so a failed refinement attempt cannot degrade a previously accepted one.

The pipeline is a directed execution graph built with LangGraph. Agents communicate through a shared, typed state object holding intermediate representations, generated code, validation results, metrics, and refinement history.

3.2 Boardwalk 2.0 Game Representation

Generated games run on Boardwalk 2.0, a redesign of the original Boardwalk framework (Becker et al. 2025) built around immutable state, needed for tree search and repeated programmatic execution. Where the original framework encapsulated gameplay in a mutable `Game` object, Boardwalk 2.0 separates state from rules: `GameState` is a snapshot of the current position, `GameRules` defines legal actions, transitions, terminal conditions, and the winner. Every transition takes a state and returns a new one, so parallel search branches never interfere with each other.

Actions are typed objects, not strings: board coordinates are `Position` objects, and legal actions are `Move` or `Placement` instances, removing string parsing and the errors that come with it. The `Board` abstraction hides its internal NumPy representation behind piece access, neighborhood traversal, directional rays, and functional updates that each return a new immutable board.

Every generated game implements five methods — `initial_state()`, `legal_moves()`, `apply()`, `is_terminal()`, `winner()` — and stores anything game-specific (reserves, repetition counters, promotion status) in `GameState`'s extensible `extra` field. Boardwalk 2.0 itself only provides state representation, board geometry, and traversal common to all games; movement, captures, promotion, scoring, and victory conditions live entirely in each generated `GameRules` subclass.

3.3 Pipeline

Five specialized agents handle specification, implementation, verification, evaluation, and refinement in sequence, exchanging artifacts through the shared state shown in Figure 1. The first four stages — rule formalization, planning, code generation, validation — make up the generation phase; once a valid implementation exists, evaluation and mutation repeat until no further improvement is accepted or a cycle limit is reached.

The pipeline's nine stages fall into three groups by how each is implemented: five are LLM-driven, generating content at temperature 0.7 — *Rule Formatter*, *Rule Validator*, *Planning Agent*, *Coding Agent*, and *Mutation Agent* (labeled *Mutator* in Figure 1); three are deterministic checks with

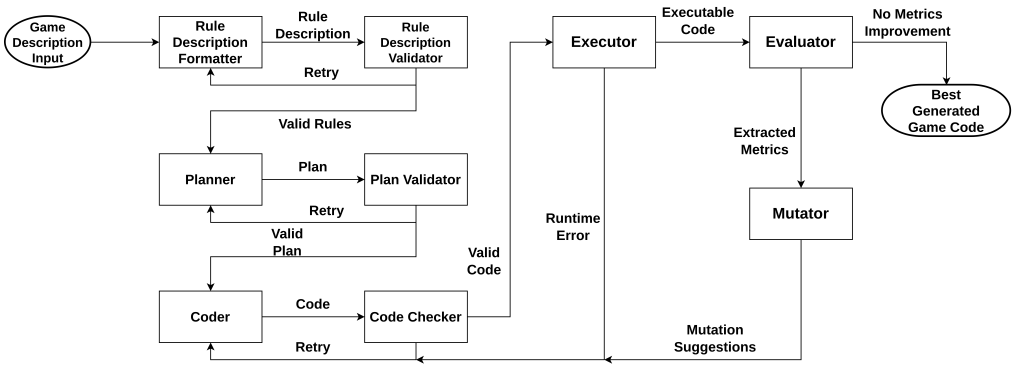


Figure 1: Overview of the proposed agentic pipeline. The generation phase transforms a natural-language description into an executable implementation through rule formalization, planning, code generation, and validation. The refinement phase repeatedly evaluates gameplay through MCTS self-play and applies metric-guided mutations until the stopping criteria are reached.

no model call — *Plan Validator*, *Code Checker*, and *Executor*, relying on fixed schema, AST, or execution checks; and one, the *Evaluator*, also makes no LLM call but is stochastic, since MCTS self-play yields varying metrics across runs.

Rule Formalization A *Rule Formatter* agent turns the prompt into a structured rulebook: board geometry, players, piece types, initial configuration, turn sequence, legal actions, captures, victory and draw conditions. It follows a fixed coordinate convention and a restricted piece alphabet, and checks board-dimension/layout consistency, piece counts, finiteness, and player symmetry before producing output.

A *Rule Validator* agent then checks structural completeness — unambiguous movement rules, well-defined terminal conditions, valid layouts, all required sections present. A failed rulebook goes back to the Rule Formatter with diagnostic feedback, for up to three iterations before generation for that prompt is abandoned.

Implementation Planning A *Planning Agent* turns a validated rulebook into a structured JSON specification — board dimensions, piece ownership, movement rules, state-transition logic, auxiliary variables, terminal conditions — rather than code directly, so the model reasons about program structure before writing it and validation has an explicit interface to check against. A *Plan Validator* then performs a deterministic structural check on the plan; invalid plans return to the Planning Agent under the same retry policy as rule formalization.

Code Generation and Validation A *Coding Agent* implements the plan as a complete `GameRules` class against the Boardwalk 2.0 API, guided by programming rules built from failure modes seen in preliminary runs: immutable state transitions, no persistent mutable class state, a consistent coordinate convention, correct turn-counter updates, valid board indexing, no accidental mutation of shared state during tree search.

Generated code passes through two checks. A *Code Checker* statically verifies syntax, confirms all required methods are present, and parses the abstract syntax tree to check that declared board dimensions match the initial layout — catching a common error without running the code. An *Executor* then loads the game in an isolated subprocess and confirms it runs without exceptions. Both checks share one retry budget per cycle; a candidate that exhausts it without producing a working implementation is discarded and treated exactly like a refinement attempt that failed to improve the current best (Section 3.6).

Gameplay Evaluation A validated candidate is evaluated through MCTS self-play with the UCT selection policy (Coulom 2006): two identical UCT agents play a fixed number of games under a per-move time budget and a turn limit. We use search-based agents rather than random play, since random policies rarely surface balance, agency, or meaningful decision-making, and identical agents on both sides mean any asymmetry comes from the rules, not player strength.

Self-play logs outcomes, game length, legal action counts, branching, and board occupancy, which we convert into the metrics below. Generated games are evaluated using the same MCTS self-play mechanism as the baseline suite (Section 3.5), providing a comparable quantitative signal to guide mutations despite operating under a lighter simulation budget.

3.4 Evaluation Metrics

We convert self-play statistics into four normalized metrics. Let N be the number of simulated games, N_d the number of decisive games, W_0 and W_1 the wins for the first and second player, T the total turns across all games, $T_{>1}$ the turns where the active player had more than one legal action, S the number of playable board positions, and $S^{(i)}_{\text{visited}}$ the number of those positions occupied at least once in game i.

Following Browne and Maire (Browne and Maire 2010), *completion* is

$$\text{completion} = \frac{N_d}{N} \tag{1}$$

and *balance*, defined only when $N_d > 0$, is

$$\text{balance} = 1 - \frac{|W_0 - W_1|}{N_d}. \tag{2}$$

Following Todd et al. (2024), *agency* is

$$\text{agency} = \frac{T_{>1}}{T}, \tag{3}$$

and we introduce *coverage*,

$$\text{coverage} = \frac{1}{N}\sum_{i=1}^{N} \frac{S^{(i)}_{\text{visited}}}{S}. \tag{4}$$

We also record branching factor, mean game length, and first-player advantage — the deviation of the first player's win rate from parity among decisive games, per Browne (Browne 2008). Unlike the first four metrics, these three are neither bounded nor strictly "higher-is-better", so they are excluded from the fitness score; instead, we report them as supplementary diagnostics.

Fitness is the harmonic mean of completion, balance, agency, and coverage, each clamped to a floor of 0.01, following Todd *et al.* (Todd et al. 2024). When $N_d = 0$, balance is undefined; we assign it the floor rather than dropping it from the mean, since dropping it would shrink the mean's term count and inflate the score of a game that never resolves relative to one that resolves but is unfair.

Algorithm 1 gives the full acceptance procedure (detailed in Section 3.6), where C represents the evaluated game candidate. It uses c, b, g, v for completion, balance, agency, and coverage, respectively, with a best subscript denoting the current best candidate's values, a relative-improvement margin θ, and a gap-closing margin α.

Algorithm 1 Candidate Evaluation and Elitist Selection

Require: C, C_{best}, f_{best}, $M_{\text{best}} = (c_{\text{best}}, b_{\text{best}}, g_{\text{best}}, v_{\text{best}})$, θ, α

Evaluate C via UCT self-play, obtaining completion c, balance b, agency g, coverage v
$c', g', v' \leftarrow \max(c, 0.01),\ \max(g, 0.01),\ \max(v, 0.01)$
$b' \leftarrow 0.01$ **if** b undefined **else** $\max(b, 0.01)$
$f \leftarrow \text{HMean}(c', b', g', v')$
if $f_{\text{best}} = -\infty$ **then**
    $C_{\text{best}}, f_{\text{best}}, M_{\text{best}} \leftarrow C, f, (c, b, g, v)$
    return true ▷ First candidate is always accepted
end if
$f_{\text{target}} \leftarrow f_{\text{best}} + \alpha(1 - f_{\text{best}})$
$\Delta_{\text{rel}} \leftarrow (f - f_{\text{best}})/f_{\text{best}}$ **if** $f_{\text{best}} > 0$ **else** $f - f_{\text{best}}$
regressed $\leftarrow (c < c_{\text{best}}) \wedge (b < b_{\text{best}})$ **if** b, b_{best} defined **else false**
gain $\leftarrow (b_{\text{best}}$ undefined$) \wedge (b$ defined$)$
if $(\text{gain} \vee f \geq f_{\text{target}} \vee \Delta_{\text{rel}} \geq \theta) \wedge \neg\,\text{regressed}$ **then**
    $C_{\text{best}}, f_{\text{best}}, M_{\text{best}} \leftarrow C, f, (c, b, g, v)$
    return true ▷ Candidate accepted as new best
else
    return false ▷ Candidate rejected – counts toward no-improvement limit
end if

3.5 Baseline Game Suite

We built a reference suite of four established games — Checkers, Connect Four, Nine Men's Morris, Othello — as a quantitative point of comparison, not as training data. Claude Sonnet 5 implemented each against the Boardwalk 2.0 API from the game's name alone, with no rule specification, mirroring how the pipeline itself receives a prompt.

Each baseline played 1,000 self-play matches between two identical UCT agents at 3 s per move, with a turn limit tuned per game so normal play could finish naturally: 130 for Checkers, 64 for Othello, 42 for Connect Four, 100 for Nine Men's Morris. Under our four-metric fitness function, the resulting scores are 0.573 (Checkers), 0.901 (Connect Four), 0.981 (Othello), 0.885 (Nine Men's Morris); Figure 2 gives the full metric breakdown.

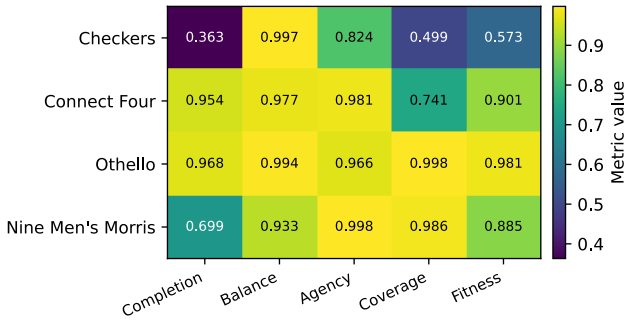


Figure 2: Gameplay metrics and aggregate fitness scores for the baseline game suite. Aggregate scores are computed using the harmonic-mean fitness function described in Section 3.4.

Chess and Tic-Tac-Toe were excluded. Chess's branching factor (27.6, against 5.8 for Checkers and 8.46 for Othello) kept UCT from searching deep enough, so games consistently hit the turn cap with degenerate metrics. Tic-Tac-Toe went the other way: every game drew under optimal UCT play, leaving balance undefined for every trial.

Beyond providing a reference point, the baseline suite tells the Mutation Agent which of a candidate's metrics deviates most from established games, focusing each refinement attempt on the largest weakness.

3.6 Metric-Guided Mutation and Elitist Refinement

A *Mutation Agent* compares the current candidate's metrics against the baseline values and targets whichever deviates most, classifying the fix as either a rule change or a code correction, with a rationale for each proposed edit. A malformed output gets up to two retries before the cycle is abandoned.

Mutations apply to the current best candidate, never to the most recent attempt, so a bad refinement can't degrade an already-accepted solution — an elitist strategy (De Jong 1975) that keeps refinement monotonic by construction. The result re-enters validation and evaluation.

A revised candidate becomes the new best only if it clears two conditions. First, either a quantitative or qualitative bar: quantitatively, we require the candidate's new fitness score f (the harmonic mean of the four metrics) to close at least a fraction α of the remaining distance to 1.0 ($f \geq f_{\text{best}} + \alpha(1 - f_{\text{best}})$, $\alpha = 0.04$) — a fixed relative-improvement bar alone breaks down near the ceiling, where even a well-refined candidate can only gain a vanishing amount. Since gap-closing alone gets too permissive as f_{best} approaches zero, a candidate also qualifies with a relative improvement of at least $\theta = 0.5$ over f_{best} (Eiben and Smith 2015); either is enough. Qualitatively, a candidate that moves from an undefined balance (no decisive games at all) to a valid, decisive state passes automatically, regardless of the resulting score. Second, it must not regress on both completion *and* balance at once, even if the score condition is met — this catches cases where the aggregate improves only because some other metric like coverage jumped, while completion and balance both got worse, which the harmonic mean doesn't always catch on its own.

A candidate failing either condition is discarded and counts toward the no-improvement limit, same as a validation failure. The loop stops after two consecutive cycles without an accepted improvement, or at a fixed cycle limit, and returns the highest-scoring candidate found.

3.7 Prompt Suite

To study how prompt detail affects both baseline quality and refinement (RQ1), we built four prompts across distinct game concepts, each at a different specification level, rather than one game varied only in detail: a connection game, a capture-by-surrounding game, an asymmetric pursuit game, a promotion game, at increasing levels of detail. Concept and detail level therefore co-vary, so differences across categories reflect both together, not detail level in isolation — a trade-off we accept here since it also exercises the pipeline across more mechanics than a single-game ladder would.

- **C1 – Minimal Prompt (connection game):** a brief concept description; board size, piece counts, and mechanics are left to the model.
- **C2 – Moderately Specified Prompt (capture game):** board dimensions, piece supply, and win/end conditions are given; movement and placement are largely unspecified.
- **C3 – Detailed Prompt (asymmetric pursuit game):** board size, piece roles and counts, per-player win conditions, turn structure, and movement are all specified.
- **C4 – Complete Rule Specification (promotion game):** a near-complete rulebook covering movement, capture, and promotion, plus a requirement to invent one supplementary mechanic — testing extension of a near-complete spec rather than interpretation of an incomplete one.

4 Experiments

4.1 Experimental Setup

We evaluate the pipeline across four language models, spanning four providers and both open- and closed-weight models. To ensure a fair baseline for evaluating multi-agent architectures, all candidates were selected to sit in a comparable reasoning tier, avoiding the imbalance of mixing high-capacity models with lightweight alternatives:

- `deepseek/deepseek-v4-pro` (DeepSeek AI)
- `openai/gpt-5` (OpenAI)
- `anthropic/claude-sonnet-5` (Anthropic)
- `qwen/qwen3.6-plus` (Alibaba)

All models are accessed through OpenRouter[1] using a single unified API, with sampling temperature fixed at 0.7 uniformly across every agent in the pipeline and across all four models, so that generation stochasticity is held constant as a variable when comparing models.

Evaluation protocol. Each candidate is evaluated via $n = 40$ simulated games of UCT self-play, with a per-move thinking budget of 1 s for both agents and a maximum turn limit of 60. These parameters differ from the baseline suite's exhaustive evaluation protocol (Section 3.5) by design; because candidates must be repeatedly evaluated on every refinement cycle, this lighter configuration provides a sufficiently robust directional signal for the mutation agent while keeping the computational and API monetary costs strictly bounded. The refinement loop terminates after two consecutive cycles without an accepted improvement (Algorithm 1) or after 10 total refinement cycles, whichever occurs first.

Infrastructure. Game simulations are parallelized via process-based parallelism, with each worker independently re-instantiating the candidate game and both UCT agents to avoid cross-process state sharing. All experiments were run locally on a Lenovo ThinkPad T480 (Intel Core i7-8650U, 4 cores / 8 threads, 32 GB RAM, Manjaro Linux) using 6 parallel worker processes.

5 Results

This section presents the experimental results across the four prompt categories introduced in Section 3.7 (C1–C4), each evaluated with all four language models under identical conditions (40 self-play games, 60-turn maximum, temperature 0.7). The section is organized around the two research questions from Section 1. We first examine how prompt specificity affects both baseline quality and refinement efficacy (RQ1). We then analyze consistency and failure patterns across the four models (RQ2).

5.1 RQ1: Prompt Specificity and Iterative Refinement Efficacy

Table 1 summarizes the final accepted candidate for every model-prompt combination, and Figure 3 shows the corresponding refinement trajectories.

C1 (Minimal Prompt, connection game): Baseline generation (Iteration 1) reliably established the core graph-traversal mechanic across all four models. The refinement phase, however, showed a strong tendency to plateau: three of four models (DeepSeek-V4-Pro, Claude-Sonnet-5,

[1] https://openrouter.ai

Prompt	Model	Score	Compl.	Bal.	Ag.	Cov.	Len.	Cap.
C1	DeepSeek-V4-Pro	0.819	0.750	0.867	0.979	0.728	35.2	0
	Claude-Sonnet-5	0.784	0.775	0.710	0.994	0.717	35.2	0
	GPT-5	0.299	0.100	1.000	1.000	0.736	59.9	36
	Qwen3.6-Plus	0.817	1.000	0.850	1.000	0.581	21.4	0
C2	DeepSeek-V4-Pro	0.010	0.000	–	0.000	0.000	0.0	0
	Claude-Sonnet-5	0.858	0.875	0.686	0.989	0.954	38.5	0
	GPT-5	0.700	0.900	0.389	0.972	0.998	39.7	4
	Qwen3.6-Plus	0.982	1.000	0.950	1.000	0.978	60.0	40
C3	DeepSeek-V4-Pro	0.038	0.500	0.000	0.696	0.443	39.9	20
	Claude-Sonnet-5	0.165	1.000	0.050	0.997	0.450	17.6	0
	GPT-5	0.038	1.000	0.000	1.000	0.406	5.8	0
	Qwen3.6-Plus	0.038	1.000	0.000	1.000	0.424	6.4	0
C4	DeepSeek-V4-Pro	0.854	0.775	0.903	0.808	0.952	48.3	9
	Claude-Sonnet-5	0.809	0.825	0.909	0.835	0.697	30.2	7
	GPT-5	0.868	0.675	0.963	0.980	0.938	40.1	13
	Qwen3.6-Plus	0.870	0.800	1.000	0.801	0.911	39.6	8

Table 1. Final accepted candidate for every model-prompt combination. Score is the harmonic-mean fitness (Algorithm 1); Compl., Bal., Ag., Cov. are completion, balance, agency, and coverage; Len. is mean game length in turns; Cap. is the number of the 40 games that reached the maximum turn limit. A dash indicates an undefined metric ($N_d = 0$).

Qwen3.6-Plus) reached their peak score within the first two mutation cycles, then spent the remaining cycles producing rejected, less-balanced variants, ending the run unable to improve upon that early peak. GPT-5 diverged sharply from this pattern: its first candidate almost never terminated decisively, and only a structural-gain acceptance in Iteration 3 — moving from an undefined balance to a valid, if still weak, decisive rate — lifted it out of that state. Average score across models on C1 was 0.680, with the widest within-category spread of any prompt except C2.

C2 (Moderately Specified Prompt, capture game): This category produced the single strongest result of the entire experiment set — Qwen3.6-Plus reached an elite fitness score after seven mutation cycles — but also its most complete failure: DeepSeek-V4-Pro's first candidate never produced a single decisive game across all simulations, and three subsequent mutation cycles failed to escape this state, ending the run at the metric floor. This is the widest score range observed for any prompt, and it occurred despite both runs starting from a structurally valid, executing implementation — the difference lies primarily in whether the generated capture mechanic was logically reachable during self-play, rather than in runtime execution failures. GPT-5 needed two rule-formalization retries for this prompt (Section 3.3's pipeline rejected a pass/no-op action incompatible with the Boardwalk 2.0 action model), the only prompt besides C3 where any model required more than one rule-formatting attempt.

C3 (Detailed Prompt, asymmetric pursuit game): Every model struggled uniformly on this category, producing the lowest scores observed across the entire benchmark. Three of four models converged on a King that reaches a corner in 5–7 moves on average with near-total first-move advantage. DeepSeek-V4-Pro's candidate exhibited the opposite failure mode: half of all games never resolved before the turn cap, rather than resolving too quickly in one player's favor. That every model, despite receiving comparatively detailed movement and win-condition specifications, converged on some variant of "the King escapes almost immediately" suggests the asymmetric pursuit mechanic itself — a single sliding piece against four blocking pieces on a small 5×5 grid — may be intrinsically difficult to balance within this pipeline's mutation budget, independent of prompt detail.

C4 (Complete Rule Specification, promotion game): This category produced the most consistent results of the four, with all models converging to a narrow score band (0.809–0.870, range 0.062, the smallest of any prompt) despite the added requirement that each model invent its own supplementary mechanic. Every model reached a stable, well-balanced implementation (balance between 0.90 and 1.00 for all four models) within four to six mutation cycles. This is consistent with C4 supplying the most complete base ruleset of the four categories: with movement, capture, and promotion fully specified, refinement only needed to tune the model's own invented addition rather than resolve ambiguity in the core mechanic itself. To illustrate the Mutation Agent's practical impact on these additions, consider the Qwen3.6-Plus run: early candidates largely failed to reach decisive outcomes, but across a few refinement cycles, the

agent identified this failure through the evaluation metrics and proposed targeted rule-level fixes—tightening the capture rules and adding a clearer endgame condition—to arrive at the balanced, decisive final game.

Taken together, average scores across categories were C4 (0.850) $>$ C1 (0.680) $>$ C2 (0.637) $>$ C3 (0.070). While the most completely specified prompt (C4) indeed produced the highest and most consistent scores, the fact that the minimal prompt (C1) outperformed both the moderately specified (C2) and detailed (C3) prompts indicates there is no strict linear relationship between prompt detail and final game quality. Instead, this reinforces the caveat noted in Section 3.7: because game concept and detail level co-vary in this prompt suite, the intrinsic difficulty of balancing specific mechanics (such as C3's asymmetric pursuit) can dominate the refinement process, overshadowing the initial level of specification detail.

5.2 RQ2: Comparison Across Language Models

Table 2 summarizes each model's fitness-score statistics alongside the failure modes observed during refinement. Qwen3.6-Plus achieved the highest mean score (0.677), narrowly ahead of Claude-Sonnet-5 (0.654); GPT-5 and DeepSeek-V4-Pro trailed (0.477 and 0.430 respectively), both pulled down by a single catastrophic run each (GPT-5 on C3, DeepSeek-V4-Pro on C2).

Every coding or execution failure observed across all 16 runs was recovered within the same refinement cycle, and no cycle exhausted its retry budget (Section 3.3) — this held even for DeepSeek-V4-Pro's C3 run, which recovered from two consecutive failures (a malformed layout assignment, then an incorrect `Board.moved()` call signature) before producing an executable candidate. This suggests that error recovery is a property of the pipeline's retry mechanism itself, rather than a capability specific to any one model. GPT-5 was the only model to require rule-formalization retries, doing so on two of its four runs (C2 and C3): in both cases, the Rule Validator rejected a rulebook specifying a pass/no-op action incompatible with the Boardwalk 2.0 action model, or leaving a dead-end board state undefined; both were corrected on the next attempt.

A second failure mode — structurally valid code that nonetheless produces a game with no reliable path to a decisive outcome — appeared once each for GPT-5 (C1, Table 1) and DeepSeek-V4-Pro (C2, all 40 games drawn). The two diverged in outcome: GPT-5 recovered via a structural-gain acceptance in its third mutation cycle (Section 3.6), while DeepSeek-V4-Pro never escaped this state across three cycles, ending at the metric floor. This suggests the acceptance criterion's structural bypass is necessary but not sufficient — escaping a degenerate state depends on the specific fix the Mutation Agent proposes, not on the acceptance rule alone.

6 Discussion

This paper introduced an agentic pipeline that generates board game mechanics as executable Python code and refines them through closed-loop, metric-guided self-play, decomposing generation into specialized agents for rule formalization, planning, coding, validation, evaluation, and mutation. A revised candidate is accepted only via a fitness gain meeting a relative-improvement or gap-closing threshold, or a qualitative transition from an unplayable to a decisively evaluable state.

Our results offer a mixed answer to RQ1: as detailed in Section 5, no strict linear relationship holds between prompt detail and outcome quality (C4 $>$ C1 $>$ C2 $>$ C3) — the minimal prompt (C1) outperformed both the moderately specified and detailed prompts (C2, C3), indicating that the underlying game concept, not only how precisely it is described, can dominate refinement difficulty. C3's asymmetric pursuit mechanic is the clearest case of this: every model converged on some variant of the same imbalanced outcome regardless of its own capability elsewhere. This is a distinction our prompt suite's design (Section 3.7) cannot fully disentangle, since game concept and detail level co-vary across C1–C4 by construction.

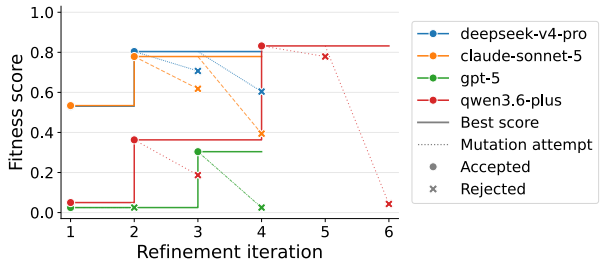
For RQ2, the pipeline exhibited consistent structural robustness across all four models: every coding or execution failure observed, including two consecutive failures within a single run, was corrected within the same refinement cycle, and no cycle across the 16 runs exhausted its retry budget (Table 2). This suggests that recovery from implementation errors is primarily a property of the error-feedback mechanism rather than a capability specific to a particular model. Outcome quality, however, varied meaningfully across models: Qwen3.6-Plus and Claude-Sonnet-5 achieved higher average scores than GPT-5 and DeepSeek-V4-Pro, while failure modes also differed — GPT-5 alone required rule-formalization retries, whereas DeepSeek-V4-Pro and Qwen3.6-Plus produced more frequent within-cycle coding corrections than Claude-Sonnet-5.

6.1 Limitations

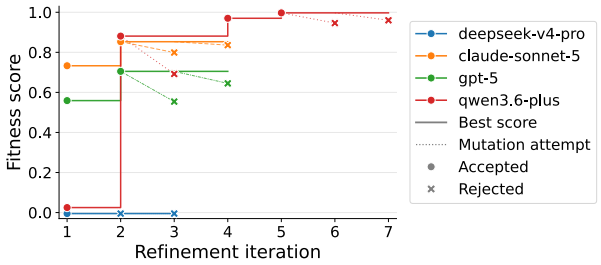
These findings carry several limitations. Each model-prompt combination was evaluated in a single run, so reported scores cannot fully separate model or prompt effects from the stochastic variance inherent to LLM generation and MCTS self-play. As noted for RQ1, the prompt suite covaries specification detail with game concept, preventing isolated causal claims about prompt length. The refinement loop also used a lighter computational budget (40 games, 60-turn cap) than the exhaustive baseline suite (Section 3.5), making generated-versus-baseline comparisons directionally informative but not strictly equivalent; the baseline suite itself covers only four games, a narrow reference for mutation. Finally, our elitist search (Algorithm 1) discards lower-scoring candidates that might otherwise serve as stepping stones, and both the acceptance criterion and the four fitness metrics — computed from identical-strength UCT self-play — remain a proxy for game quality rather than ground truth: they cannot detect structural derivativeness, nor reveal how a game behaves across varying levels of human skill.

6.2 Future Work

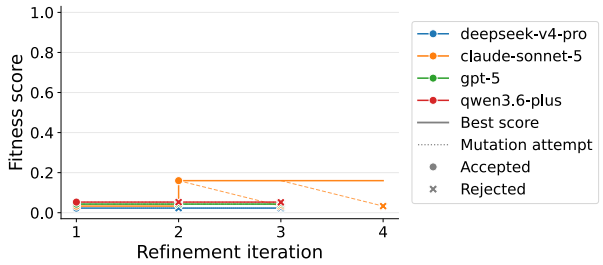
Future work should prioritize repeated trials to establish confidence intervals, and a prompt suite that holds game concept fixed while varying only specification detail. The baseline suite should expand to cover broader me-



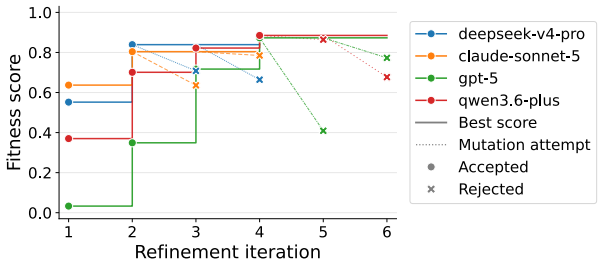
(a) C1 (Connection game)



(b) C2 (Capture game)



(c) C3 (Asymmetric pursuit game)



(d) C4 (Promotion game)

Figure 3: Evolution of the incumbent game's fitness score across refinement iterations for all four prompt categories, across all four evaluated models. Iteration 1 shows the initial from-scratch candidate's evaluation. Solid lines track the current best candidate's score. Dashed lines represent mutation attempts branching from the current incumbent. Circles (•) denote mutations accepted as the new best, while crosses (×) denote rejected mutations. The refinement loop terminates when an incumbent exhausts its budget of two consecutive rejected mutations.

Model	Mean Score	Min	Max	Coding/Exec Failures	Rule-Formalization Retries	Degenerate-Outcome Runs
Qwen3.6-Plus	0.677	0.038	0.982	4	0	0
Claude-Sonnet-5	0.654	0.165	0.858	1	0	0
GPT-5	0.477	0.038	0.868	0	2	1 (Recovered)
DeepSeek-V4-Pro	0.430	0.010	0.854	4	0	1 (Unrecovered)

Table 2. Per-model summary across all four prompts: fitness-score statistics and failure-mode counts observed during refinement (RQ2).

chanic families, and adaptive mutation budgets could better handle complex rulesets. Replacing single-elite selection with population- or archive-based strategies such as MAP-Elites (Todd et al. 2024) would avoid permanently discarding candidates that could serve as useful stepping stones.

Evaluation should also move beyond identical-agent self-play: multi-skill MCTS agents would let us assess the strategy ladder and theory-of-fun properties our current setup isolates out by design, while structural novelty checks against Ludii's corpus would help detect derivativeness. Ultimately, these automated checks should be validated against systematic human playtesting to confirm that metric-guided improvements produce rulesets people find coherent, fair, and enjoyable.

Acknowledgments

The first author acknowledges the Fundação de Amparo à Pesquisa do Estado do Rio Grande do Sul (FAPERGS) for scholarship support received during the period in which this research was conducted. This work was supported by Kunumi Institute. The authors thank the institution for its financial support and commitment to advancing scientific research. This study was financed in part by the Coordenação de Aperfeiçoamento de Pessoal de Nível Superior - Brasil (CAPES) - Finance Code 001.

References

Anjum, A.; Li, Y.; and Togelius, J. 2024. The Ink Splotch Effect: A Case Study on ChatGPT as a Co-Creative Game Designer. In *International Conference on the Foundations of Digital Games*.

Becker, A. G.; Oliveira, G. B. d.; Rossato, L. B.; and Tavares, A. R. 2025. Boardwalk: Towards a Framework for Creating Board Games with LLMs. In *Anais do XXIV Simpósio Brasileiro de Jogos e Entretenimento Digital (SBGames 2025)*, SBGames 2025, 655–667. Sociedade Brasileira de Computação.

Beyer, M.; Agureikin, A.; Anokhin, A.; Laenger, C.; Nolte, F.; Winterberg, J.; Renka, M.; Rieger, M.; Pflanzl, N.; Preuss, M.; and Volz, V. 2016. An Integrated Process for Game Balancing.

Browne, C. 2008. *Evolutionary Game Design*. Ph.D. thesis, Queensland University of Technology.

Browne, C.; and Maire, F. 2010. Evolutionary Game Design. *IEEE Transactions on Computational Intelligence and AI in Games*, 2(1): 1–16.

Campbell, M.; Jr., A. J. H.; and hsiung Hsu, F. 2002. Deep Blue. *Artificial Intelligence*, 134(1–2): 57–83.

Chen, D.; Zhang, H.; Wang, H.; Huo, Y.; Li, Y.; and Wang, J. 2023. GameGPT: Multi-agent Collaborative Framework for Game Development. arXiv:2310.08067.

Cook, M. 2025. Pixie: Code-Level Mechanic Generation for Game Designers. In *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment*, volume 21, 206–215.

Cook, M.; Colton, S.; Raad, A.; and Gow, J. 2013. Mechanic Miner: Reflection-Driven Game Mechanic Discovery and Level Design. In *European Conference on the Applications of Evolutionary Computation*, 284–293. Springer.

Coulom, R. 2006. Efficient Selectivity and Backup Operators in Monte-Carlo Tree Search. In *International Conference on Computers and Games*, 72–83. Springer.

De Jong, K. A. 1975. *An Analysis of the Behavior of a Class of Genetic Adaptive Systems*. Ph.D. thesis, University of Michigan, Ann Arbor, MI. University Microfilms No. 76-9381.

de Mesentier Silva, F.; Lee, S.; Togelius, J.; and Nealen, A. 2017. AI-based Playtesting of Contemporary Board Games. In *Proceedings of the 12th International Conference on the Foundations of Digital Games*. ACM.

Eberhardinger, M.; Goodman, J.; Dockhorn, A.; Perez-Liebana, D.; Gaina, R. D.; Çakmak, D.; Maghsudi, S.; and Lucas, S. 2025. From Code to Play: Benchmarking Program Search for Games Using Large Language Models. *IEEE Transactions on Games*, 1–13.

Ebner, M.; Levine, J.; Lucas, S. M.; Schaul, T.; Thompson, T.; and Togelius, J. 2013. Towards a Video Game Description Language. In *Dagstuhl Follow-Ups*, volume 6.

Eiben, A. E.; and Smith, J. E. 2015. *Introduction to Evolutionary Computing*. Berlin, Heidelberg: Springer, 2nd edition.

Gaina, R. D.; Balla, M.; Dockhorn, A.; Montoliu, R.; and Perez-Liebana, D. 2020. TAG: A Tabletop Games Framework. In *Experimental AI in Games (EXAG), AIIDE 2020 Workshop*.

Goodman, J.; Wallat, A.; Perez-Liebana, D.; and Lucas, S. 2023. A case study in AI-assisted board game design. In *2023 IEEE Conference on Games (CoG)*, 1–4.

Guerrero-Romero, C.; Lucas, S. M.; and Perez-Liebana, D. 2018. Using a Team of General AI Algorithms to Assist Game Design and Testing. In *2018 IEEE Conference on Computational Intelligence and Games (CIG)*, 1–8. IEEE Press.

Hom, V.; and Marks, J. 2007. Automatic Design of Balanced Board Games. In *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment*, volume 3, 25–30.

Hong, S.; Zhuge, M.; Chen, J.; et al. 2024. MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework. In *International Conference on Learning Representations (ICLR)*.

Hu, C.; Zhao, Y.; and Liu, J. 2024. Game Generation via Large Language Models. In *2024 IEEE Conference on Games (CoG)*, 1–4.

Jiang, J.; Wang, F.; Shen, J.; Kim, S.; and Kim, S. 2024. A Survey on Large Language Models for Code Generation. *ACM Transactions on Software Engineering and Methodology*, 35(2): 1–72.

Jiang, Z.; Earle, S.; Khalifa, A.; and Togelius, J. 2026. Agentic PCG: Procedural Content Generation via Tool-using LLMs. *SSRN Preprint*.

Khalifa, A.; Green, M. C.; Perez-Liebana, D.; and Togelius, J. 2017. General video game rule generation. *2017 IEEE Conference on Computational Intelligence and Games (CIG)*, 170–177.

Lanzi, P. L.; and Loiacono, D. 2023. ChatGPT and Other Large Language Models as Evolutionary Engines for Online Interactive Collaborative Game Design. *ACM Digital Library*.

Li, G.; Hammoud, H. A. A. K.; Itani, H.; Khizbullin, D.; and Ghanem, B. 2023. CAMEL: Communicative Agents for Mind Exploration of Large Language Model Society. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Lyman, B.; Ebrahimi, A.; Cox, J.; Chan, S.; Barney, C.; and De Schutter, B. 2024. Cardistry: Exploring a GPT Model Workflow as an Adapted Method of Gaminiscing. In *Proceedings of the 19th International Conference on the Foundations of Digital Games (FDG)*. ACM.

Perez-Liebana, D.; Samothrakis, S.; Togelius, J.; Schaul, T.; Lucas, S. M.; Couetoux, A.; Lee, J.; Lim, C.-U.; and Thompson, T. 2016. The 2014 General Video Game Playing Competition. *IEEE Transactions on Computational Intelligence and AI in Games*, 8: 229–243.

Piette, E.; Soemers, D.; Stephenson, M.; Sironi, C.; Winands, M.; and Browne, C. 2020. *Ludii - The Ludemic*

General Game System, volume 325 of *Frontiers in Artificial Intelligence and Applications*, 411–418. Netherlands: IOS Press. Funding Information: This research is part of the European Research Council-funded Digital Ludeme Project (ERC Consolidator Grant #771292). This work is partially supported by The Netherlands Organisation for Scientific Research (NWO) in the framework of the GoGeneral Project (Grant No. 621.001.121). Publisher Copyright: © 2020 The authors and IOS Press.; 24th European Conference on Artificial Intelligence, ECAI 2020 ; Conference date: 29-08-2020 Through 05-09-2020.

Qian, C.; Cong, X.; Yang, C.; Chen, W.; Su, Y.; Xu, J.; Liu, Z.; and Sun, M. 2024. ChatDev: Communicative Agents for Software Development. In *Annual Meeting of the Association for Computational Linguistics (ACL)*.

Schaul, T. 2013. A video game description language for model-based or interactive learning. In *IEEE Conference on Computational Intelligence in Games (CIG)*, 1–8.

Shannon, C. E. 1950. Programming a Computer for Playing Chess. *Philosophical Magazine*, 41(314): 256–275.

Silver, D.; Huang, A.; Maddison, C. J.; Guez, A.; Sifre, L.; van den Driessche, G.; Schrittwieser, J.; Antonoglou, I.; Panneershelvam, V.; Lanctot, M.; et al. 2016. Mastering the Game of Go with Deep Neural Networks and Tree Search. *Nature*, 529(7587): 484–489.

Silver, D.; Schrittwieser, J.; Simonyan, K.; Antonoglou, I.; Huang, A.; Guez, A.; Hubert, T.; Baker, L.; Lai, M.; Bolton, A.; et al. 2018. A General Reinforcement Learning Algorithm that Masters Chess, Shogi, and Go Through Self-Play. *Science*, 362(6419): 1140–1144.

Sudhakaran, S.; González-Duque, M.; Freiberger, M.; Glanois, C.; Najarro, E.; and Risi, S. 2023. MarioGPT: Open-Ended Text2Level Generation through Large Language Models. In Oh, A.; Naumann, T.; Globerson, A.; Saenko, K.; Hardt, M.; and Levine, S., eds., *Advances in Neural Information Processing Systems*, volume 36, 54213–54227. Curran Associates, Inc.

Tanaka, T.; and Simo-Serra, E. 2024. Grammar-Based Game Description Generation Using Large Language Models. *IEEE Transactions on Games*, 18(1): 30–43.

Todd, G.; Earle, S.; Nasir, M. U.; Green, M. C.; and Togelius, J. 2023. Level Generation Through Large Language Models. In *Proceedings of the 18th International Conference on the Foundations of Digital Games*, FDG 2023, 1–8. ACM.

Todd, G.; Padula, A. G.; Stephenson, M.; Piette, E.; Soemers, D. J.; and Togelius, J. 2024. GAVEL: Generating Games via Evolution and Language Models. In *Advances in Neural Information Processing Systems*, volume 37, 110723–110745. Curran Associates, Inc.

Torii, M. G.; Murakami, T.; and Ochiai, Y. 2023. Lottery and Sprint: Generate a Board Game with Design Sprint Method on AutoGPT. In *Companion Proceedings of the Annual Symposium on Computer-Human Interaction in Play (CHI PLAY Companion '23)*, 259–265. ACM.

Wang, L.; Ma, C.; Zhang, W.; et al. 2025. A Survey on Large Language Model based Autonomous Agents. *Frontiers of Computer Science*, 19(1): 191418.

Wu, Q.; Bansal, G.; Zhang, J.; Wu, Y.; Li, S.; Zhu, E.; Jiang, L.; Zhang, X.; Wang, C.; et al. 2024. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. In *Conference on Language Modeling (COLM)*.

Xi, Z.; Chen, W.; Guo, X.; Wang, W.; et al. 2023. The Rise and Potential of Large Language Model Based Agents: A Survey. *arXiv preprint arXiv:2309.07864*.

Yang, D.; Kleinman, E.; and Harteveld, C. 2024. GPT for games: A scoping review (2020-2023). In *2024 IEEE Conference on Games (CoG)*, 1–8. IEEE.

Yang, D.; Kleinman, E.; and Harteveld, C. 2025. GPT for Games: An Updated Scoping Review (2020-2024). *IEEE Transactions on Games*, 1–16.

Zammit, M.; and Liapis, A. 2024. CrawLLM: Theming Games with Large Language Models. In *Proceedings of the IEEE Conference on Games (CoG)*.

Zhao, Y.; Borovikov, I.; de Mesentier Silva, F.; Beirami, A.; Rupert, J.; Somers, C.; Harder, J.; Kolen, J.; Pinto, J.; Pourabolghasem, R.; Pestrak, J.; Chaput, H.; Sardari, M.; Lin, L.; Narravula, S.; Aghdaie, N.; and Zaman, K. 2020. Winning Isn't Everything: Enhancing Game Development with Intelligent Agents. arXiv:1903.10545.